

Hướng dẫn Unit Test với JUnit 5

Tài liệu hướng dẫn step-by-step • Lý thuyết và thực hành

• by tralTB@fe.edu.vn

Mục lục

Phần I – Lý thuyết nền tảng.....	3
1.1. Unit Testing là gì?	3
1.2. Giới thiệu JUnit 5	3
1.3. AAA Pattern (Arrange – Act – Assert)	4
1.4. Maven POM (Project Object Model)	5
Phần II – Chuẩn bị môi trường	6
Bước 1: Cài đặt JDK 17 (hoặc tải phiên bản mới hơn jdk 21, 25, rồi cấu hình file pom cho phù hợp)	6
Bước 2: Cài đặt IntelliJ IDEA (khuyến nghị 2022+)	6
Bước 3: Cài đặt Maven (tùy chọn – IntelliJ đã có sẵn)	6
Phần III – Thực hành 1: DemoCalculator	6
Bước 1: Tạo project mới trong IntelliJ	7
Bước 2: Cấu hình pom.xml	7
Bước 3: Tạo class cần kiểm thử	8
Bước 4: Viết test class theo AAA Pattern	8
Bước 5: Chạy test trong IntelliJ	10
Phần IV – Thực hành 2: Parameterized Test với CSV	10
Bước 1: Đảm bảo đã có dependency junit-jupiter-params	10
Bước 2: Bổ sung method multiply vào Calculator	10
Bước 3: Tạo file dữ liệu src/test/resources/data.csv	10
Bước 4: Viết Parameterized Test theo AAA	10
Bước 5: Chạy	11
Bổ sung – Các nguồn dữ liệu khác	11
Phần V – Thực hành 3: Lifecycle Annotations	11
Mô hình vòng đời	11

Code minh hoạ.....	12
Phần VI – Bài tập áp dụng: InsuranceClaim	13
Bước 1: Tạo class InsuranceClaim ở src/main/java/tralTB/example/	13
Bước 2: Viết InsuranceClaimTest theo AAA.....	14
Bước 3: Chạy mvn test hoặc Run trong IntelliJ → tất cả test phải Pass 	17
Phần VII – Bài tập nộp: AccountService	17
7.1. Yêu cầu Lab 2: Viết unit test cho chức năng Đăng ký tài khoản (Sẽ nộp bài trên Github classroom, chấm điểm tự động) Tham khảo trước nội dung:	17
7.2. Class AccountService	17
7.3. File dữ liệu test	18
7.4. Class test AccountServiceTest	18
7.5. Tiêu chí chấm điểm gợi ý	20
7.6. (Khuyến nghị) Thêm JaCoCo để đo coverage	20
Phần VIII – Best Practices & Checklist	21
8.1. Bảng tổng kết lưu ý khi lập trình JUnit	21
8.2. Checklist trước khi nộp Lab 2	21

Phần I – Lý thuyết nền tảng

1.1. Unit Testing là gì?

Unit Testing (kiểm thử đơn vị) là cấp độ kiểm thử thấp nhất trong mô hình V-Model, trong đó từng "đơn vị" mã nguồn (thường là một **method** hoặc một **class**) được kiểm thử **độc lập** với phần còn lại của hệ thống.

Đặc điểm của một Unit Test tốt:

Tiêu chí	Ý nghĩa
Fast	Chạy nhanh (< 1s mỗi test)
Independent	Không phụ thuộc test khác, có thể chạy theo bất kỳ thứ tự nào
Repeatable	Cùng input → cùng output, bất kể môi trường
Self-validating	Tự xác định Pass/Fail, không cần người đọc log
Timely	Viết cùng lúc (hoặc trước) với production code (TDD)

Năm tiêu chí trên thường được nhớ bằng từ viết tắt **F.I.R.S.T.**

Lợi ích của Unit Testing:

- Phát hiện lỗi sớm → giảm chi phí sửa lỗi (theo Boehm, lỗi phát hiện ở giai đoạn unit rẻ hơn ~10–100 lần so với giai đoạn release).
- Là "tài liệu sống" mô tả cách dùng class/method.
- Hỗ trợ refactor an toàn: bất kỳ thay đổi nào phá hành vi đều bị test phát hiện.
- Là điều kiện tiên quyết để áp dụng **CI/CD** (Continuous Integration / Continuous Delivery).

1.2. Giới thiệu JUnit 5

JUnit 5 là framework kiểm thử mặc định cho Java hiện đại, gồm 3 module chính:

JUnit 5 = JUnit Platform + JUnit Jupiter + JUnit Vintage

- **JUnit Platform:** nền tảng để chạy test trên JVM (tích hợp với Maven, Gradle, IDE...).
- **JUnit Jupiter:** API và Engine mới của JUnit 5 (các annotation `@Test`, `@BeforeEach`, ...).

- **JUnit Vintage:** cho phép chạy lại test cũ viết bằng JUnit 3/4.

Các annotation và assertion thường dùng:

Annotation / API	Vai trò
@Test	Đánh dấu một phương thức là test case
@BeforeEach / @AfterEach	Chạy trước/sau từng test method
@BeforeAll / @AfterAll	Chạy một lần duy nhất trước/sau toàn bộ class (phải static)
@DisplayName("...")	Đặt tên hiển thị dễ đọc cho test case
@Disabled	Tạm thời tắt một test
@ParameterizedTest	Test với nhiều bộ dữ liệu khác nhau
@CsvSource, @CsvFileSource, @ValueSource	Nguồn dữ liệu cho @ParameterizedTest
assertEquals(expected, actual)	So sánh giá trị mong đợi và thực tế
assertTrue / assertFalse	Kiểm tra biểu thức boolean
assertThrows(Ex.class, () -> ...)	Mong đợi một exception được ném ra
assertAll(...)	Gom nhiều assertion, chạy hết kể cả khi có lỗi
assertNull / assertNotNull	Kiểm tra null

1.3. AAA Pattern (Arrange – Act – Assert)

AAA là mẫu thiết kế test phổ biến nhất, chia mỗi test thành **3 khối rõ ràng**:

Bước	Mục đích	Ví dụ
Arrange (Sắp đặt)	Khởi tạo đối tượng, dữ liệu đầu vào, trạng thái ban đầu	Calculator calc = new Calculator(); int a = 2, b = 3;

Bước	Mục đích	Ví dụ
Act (Hành động)	Gọi phương thức cần kiểm thử – chỉ 1 hành động duy nhất	<code>int result = calc.add(a, b);</code>
Assert (Khẳng định)	Kiểm tra kết quả thực tế so với mong đợi	<code>assertEquals(5, result);</code>

Mẫu code chuẩn AAA:

```

@Test
@DisplayName("add(2, 3) should return 5")
void testAdd_TwoPositiveNumbers_ReturnsSum() {
    // Arrange
    Calculator calculator = new Calculator();
    int a = 2;
    int b = 3;
    int expected = 5;

    // Act
    int actual = calculator.add(a, b);

    // Assert
    assertEquals(expected, actual, "2 + 3 phải bằng 5");
}

```

Lưu ý khi viết AAA:

- Mỗi test **chỉ kiểm thử một hành vi (one behavior per test)**.
- Để comment // Arrange, // Act, // Assert (hoặc dòng trắng) giúp người đọc dễ nhận ra cấu trúc.
- Đặt tên test theo công thức: `methodUnderTest_StateUnderTest_ExpectedBehavior` (vd: `divide_ByZero_ThrowsIllegalArgumentException`).

1.4. Maven POM (Project Object Model)

POM là file `pom.xml` – trái tim của dự án Maven, mô tả:

- Toạ độ dự án (`groupId`, `artifactId`, `version`).
- Cấu hình JDK, encoding.
- **Dependencies** (thư viện phụ thuộc – bao gồm JUnit).
- **Plugins** (vd: `maven-surefire-plugin` để chạy test).

Cấu trúc thư mục chuẩn Maven:

```
DemoCalculator/
```

```
├── pom.xml
└── src/
    ├── main/
    │   └── java/
    │       └── tralTB/example/
    │           └── Calculator.java
    └── test/
        ├── java/
        │   └── tralTB/example/
        │       └── CalculatorTest.java
        └── resources/
            └── data.csv
```

Quy ước: code production trong src/main/java, code test trong src/test/java, dữ liệu test trong src/test/resources.

Phần II – Chuẩn bị môi trường

Bước 1: Cài đặt JDK 17 (hoặc tải phiên bản mới hơn jdk 21, 25, rồi cấu hình file pom cho phù hợp)

Tải tại: <https://www.oracle.com/java/technologies/javase/jdk17-archive-downloads.html>

Tại sao chọn JDK 17?

- Là phiên bản LTS (Long-Term Support).
- Tương thích tốt với junit-jupiter 5.x, maven-surefire-plugin $\geq 2.22.2$, jacoco-maven-plugin $\geq 0.8.5$.

Sau khi cài đặt:

- Đặt biến môi trường JAVA_HOME trỏ tới thư mục cài JDK.
- Thêm %JAVA_HOME%\bin vào PATH.
- Kiểm tra bằng lệnh:

```
java -version
```

Kết quả mong đợi: java version "17.x.x".

Bước 2: Cài đặt IntelliJ IDEA (khuyến nghị 2022+)

IntelliJ IDEA Community Edition là đủ cho lab này. Tải tại <https://www.jetbrains.com/idea/download/>.

Bước 3: Cài đặt Maven (tùy chọn – IntelliJ đã có sẵn)

Có thể dùng Maven Wrapper (mvnw) đi kèm dự án, không cần cài Maven riêng.

Phần III – Thực hành 1: DemoCalculator

Mục tiêu: Tạo project Maven, viết một class Calculator đơn giản và unit test cho nó theo AAA Pattern.

Bước 1: Tạo project mới trong IntelliJ

File → New → Project:

- **Name:** DemoCalculator
- **Location:** chọn thư mục lưu
- **Build system:** Maven
- **JDK:** chọn JDK 17
- **GroupId:** ghi tên (vd: tralTB.example)
- **ArtifactId:** DemoCalculator

Bước 2: Cấu hình pom.xml

Mở file pom.xml ở thư mục gốc, thay nội dung bằng cấu hình sau (phiên bản JUnit Jupiter 5.12.2):

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>tralTB.example</groupId>
  <artifactId>DemoCalculator</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <!-- JUnit Jupiter Engine -->
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-engine</artifactId>
      <version>5.12.2</version>
      <scope>test</scope>
    </dependency>

    <!-- JUnit Jupiter API -->
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter-api</artifactId>
      <version>5.12.2</version>
```

```
<scope>test</scope>
</dependency>

<!-- JUnit Jupiter Params (dùng cho @ParameterizedTest) -->
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-params</artifactId>
  <version>5.12.2</version>
  <scope>test</scope>
</dependency>
</dependencies>

<build>
  <plugins>
    <!-- Plugin chạy unit test với Maven -->
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
    </plugin>
  </plugins>
</build>
</project>
```

Sau đó nhấn biểu tượng **"Sync Maven Changes"** (chữ M nhỏ phía góc phải) hoặc Ctrl+Shift+O để Maven tải dependencies. Khi tab **Maven** ở bên phải đã hiển thị các dependencies thì cấu hình thành công.

Tip: Có thể tra cứu phiên bản mới nhất tại <https://mvnrepository.com/artifact/org.junit.jupiter/junit-jupiter-engine>.

Bước 3: Tạo class cần kiểm thử

Tạo file src/main/java/tralTB/example/Calculator.java:

```
package tralTB.example;

public class Calculator {

    public int add(int a, int b) {
        return a + b;
    }

    public int divide(int a, int b) {
        if (b == 0) {
            throw new IllegalArgumentException("Cannot divide by zero");
        }
        return a / b;
    }
}
```

Bước 4: Viết test class theo AAA Pattern

Tạo file src/test/java/tralTB/example/CalculatorTest.java:

```
package tralTB.example;
```

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;

class CalculatorTest {

    private final Calculator calculator = new Calculator();

    @Test
    @DisplayName("add(2,3) trả về 5")
    void add_TwoPositiveNumbers_ReturnsSum() {
        // Arrange
        int a = 2;
        int b = 3;
        int expected = 5;

        // Act
        int actual = calculator.add(a, b);

        // Assert
        assertEquals(expected, actual, "2 + 3 phải bằng 5");
    }

    @Test
    @DisplayName("divide(6,3) trả về 2")
    void divide_ValidDivision_ReturnsQuotient() {
        // Arrange
        int a = 6;
        int b = 3;

        // Act
        int result = calculator.divide(a, b);

        // Assert
        assertEquals(2, result);
    }

    @Test
    @DisplayName("divide(10,0) ném IllegalArgumentException")
    void divide_ByZero_ThrowsIllegalArgumentException() {
        // Arrange
        int a = 10;
        int b = 0;

        // Act & Assert
        IllegalArgumentException ex = assertThrows(
            IllegalArgumentException.class,
            () -> calculator.divide(a, b)
        );
        assertEquals("Cannot divide by zero", ex.getMessage());
    }
}
```

Bước 5: Chạy test trong IntelliJ

- **Cách 1:** Chuột phải vào class CalculatorTest → **Run 'CalculatorTest'**.
- **Cách 2:** Bấm icon  bên trái tên method.
- **Cách 3:** Chạy bằng Maven CLI:

```
mvn test
```

Kết quả thành công sẽ hiển thị 3 dấu  trong tab **Run** của IntelliJ. Nếu có test fail sẽ hiển thị  kèm stack trace.

Phần IV – Thực hành 2: Parameterized Test với CSV

Mục tiêu: Sử dụng `@ParameterizedTest` + `@CsvFileSource` để chạy **một** test với **nhiều** bộ dữ liệu khác nhau, tránh phải copy-paste test.

Bước 1: Đảm bảo đã có dependency junit-jupiter-params

(Đã thêm trong pom.xml ở Phần III.)

Bước 2: Bổ sung method multiply vào Calculator

```
public int multiply(int a, int b) {  
    return a * b;  
}
```

Bước 3: Tạo file dữ liệu src/test/resources/data.csv

```
a,b,expected  
2,3,6  
0,5,0  
-1,4,-4  
-3,-3,9  
7,1,7
```

Lưu ý: trong IntelliJ, click chuột phải vào thư mục test → New → Directory → resources. Sau đó tạo file data.csv bên trong.

Bước 4: Viết Parameterized Test theo AAA

Bổ sung vào CalculatorTest:

```
import org.junit.jupiter.params.ParameterizedTest;  
import org.junit.jupiter.params.provider.CsvFileSource;  
  
@ParameterizedTest(name = "Test {index} => {0} * {1} = {2}")  
@CsvFileSource(resources = "/data.csv", numLinesToSkip = 1)  
@DisplayName("multiply: kiểm thử với nhiều bộ dữ liệu từ CSV")  
void multiply_VariousInputs_ReturnsProduct(int a, int b, int expected) {  
    // Arrange: a, b, expected do JUnit inject từ CSV  
  
    // Act
```

```

int actual = calculator.multiply(a, b);

// Assert
assertEquals(expected, actual,
    () -> a + " * " + b + " phải bằng " + expected);
}

```

numLinesToSkip = 1 để bỏ qua dòng tiêu đề. Trong tài liệu gốc dùng numLinesToSkip = 0 (không có header) – tùy chọn theo dữ liệu thực tế.

Bước 5: Chạy

Chuột phải multiply_VariousInputs_ReturnsProduct → **Run**. JUnit sẽ tự sinh **5 test instance**, mỗi instance là một dòng CSV.

Bổ sung – Các nguồn dữ liệu khác

Annotation	Mô tả	Ví dụ
@ValueSource(ints = {1,2,3})	Truyền 1 tham số đơn	Test với từng số nguyên
@CsvSource({"2,3,6", "0,5,0"})	Dữ liệu inline	Khi dữ liệu ngắn
@CsvFileSource(resources = "/data.csv")	Dữ liệu từ file CSV	Khi nhiều dòng
@MethodSource("provider")	Lấy từ static method trả về Stream<Arguments>	Khi cần object phức tạp
@EnumSource(MyEnum.class)	Duyệt qua các giá trị enum	Khi test theo enum

Phần V – Thực hành 3: Lifecycle Annotations

Mục tiêu: Hiểu vòng đời của một test class với @BeforeAll, @AfterAll, @BeforeEach, @AfterEach.

Mô hình vòng đời

```

@BeforeAll      <- chạy 1 lần, trước tất cả các test (static)
├── @BeforeEach <- trước mỗi test
│   ├── @Test #1
│   └── @AfterEach <- sau mỗi test
└── @BeforeEach

```

```
    @Test #2
    @AfterEach
    ...
@AfterAll          <- chạy 1 lần, sau tất cả các test (static)
```

Code minh họa

```
package tralTB.example;

import org.junit.jupiter.api.*;

import static org.junit.jupiter.api.Assertions.assertEquals;

class CalculatorLifecycleTest {

    static Calculator calculator;

    @BeforeAll
    static void initAll() {
        System.out.println("==> Khởi tạo tài nguyên dùng chung");
        calculator = new Calculator();
    }

    @AfterAll
    static void cleanupAll() {
        System.out.println("==> Giải phóng tài nguyên dùng chung");
        calculator = null;
    }

    @BeforeEach
    void setUp(TestInfo info) {
        System.out.println("---> Bắt đầu test: " + info.getDisplayName());
    }

    @AfterEach
    void tearDown(TestInfo info) {
        System.out.println("---> Kết thúc test: " + info.getDisplayName());
    }

    @Test
    @DisplayName("Kiểm tra phép cộng với hai số dương")
    void testAddition() {
        // Arrange
        int a = 2, b = 3;

        // Act
        int result = calculator.add(a, b);

        // Assert
        assertEquals(5, result, "Addition should return 5");
    }

    @Test
    @DisplayName("Kiểm tra phép chia hợp lệ")
    void testDivide() {
        // Arrange
        int a = 6, b = 3;
```

```

        // Act
        int result = calculator.divide(a, b);

        // Assert
        assertEquals(2, result);
    }
}

```

Khi nào dùng cái nào?

Tình huống	Dùng
Khởi tạo tài nguyên tốn kém (kết nối DB, đọc file lớn) dùng chung	@BeforeAll
Reset trạng thái object cho mỗi test (đảm bảo độc lập)	@BeforeEach
Đóng connection, xoá file tạm sau khi tất cả test xong	@AfterAll
Log/ghi audit theo từng test	@AfterEach

Phần VI – Bài tập áp dụng: InsuranceClaim

Viết unit test cho class InsuranceClaim bằng JUnit 5.

Bước 1: Tạo class InsuranceClaim ở src/main/java/tralTB/example/

```

package tralTB.example;

public class InsuranceClaim {

    private final String claimId;
    private double amount;
    private String claimStatus;

    public InsuranceClaim(String id, double claimAmount) {
        if (id == null || id.isEmpty()) {
            throw new IllegalArgumentException("Claim ID cannot be null or empty");
        }
        if (claimAmount <= 0) {
            throw new IllegalArgumentException("Amount must be positive");
        }
        this.claimId = id;
        this.amount = claimAmount;
        this.claimStatus = "Pending";
    }

    public boolean processClaim(String newStatus) {
        if (newStatus == null) {

```

```
        throw new IllegalArgumentException("Status cannot be null");
    }
    if (!"Pending".equals(this.claimStatus)) {
        return false;
    }
    this.claimStatus = newStatus;
    return true;
}

public double calculatePayout() {
    if ("Approved".equals(claimStatus)) {
        return amount * 0.85;    // trừ 15% phí
    }
    return 0;
}

public void updateClaimAmount(double newAmount) {
    if (newAmount <= 0) {
        throw new IllegalArgumentException("Amount must be positive");
    }
    this.amount = newAmount;
}

public String getClaimId() { return claimId; }
public double getAmount() { return amount; }
public String getClaimStatus() { return claimStatus; }

@Override
public String toString() {
    return "InsuranceClaim{" +
        "claimId='" + claimId + '\'' +
        ", amount=" + amount +
        ", claimStatus='" + claimStatus + '\'' +
        '}';
}
}
```

Bước 2: Viết InsuranceClaimTest theo AAA

```
package tralTB.example;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

import static org.junit.jupiter.api.Assertions.*;

class InsuranceClaimTest {

    private InsuranceClaim claim;

    @BeforeEach
    void setUp() {
        // Arrange chung cho mọi test
        claim = new InsuranceClaim("C001", 1000.0);
    }
}
```

```
}

@Test
@DisplayName("Constructor khởi tạo đúng giá trị")
void constructor_ValidInput_InitializesValues() {
    // Assert
    assertEquals("C001", claim.getClaimId());
    assertEquals(1000.0, claim.getAmount());
    assertEquals("Pending", claim.getClaimStatus());
}

@Test
@DisplayName("Constructor ném exception khi amount không hợp lệ")
void constructor_InvalidAmount_Throws() {
    // Act & Assert
    assertThrows(IllegalArgumentException.class,
        () -> new InsuranceClaim("C002", -500));
}

@Test
@DisplayName("Constructor ném exception khi claimId null")
void constructor_NullClaimId_Throws() {
    assertThrows(IllegalArgumentException.class,
        () -> new InsuranceClaim(null, 1000.0));
}

@Test
@DisplayName("processClaim cập nhật status khi đang Pending")
void processClaim_StatusPending_UpdatesStatus() {
    // Act
    boolean result = claim.processClaim("Approved");

    // Assert
    assertTrue(result);
    assertEquals("Approved", claim.getClaimStatus());
}

@Test
@DisplayName("processClaim trả về false khi đã không còn Pending")
void processClaim_StatusNotPending_ReturnsFalse() {
    // Arrange
    claim.processClaim("Approved");

    // Act
    boolean result = claim.processClaim("Rejected");

    // Assert
    assertFalse(result);
    assertEquals("Approved", claim.getClaimStatus());
}

@Test
@DisplayName("processClaim ném exception khi truyền null")
void processClaim_NullInput_Throws() {
    assertThrows(IllegalArgumentException.class,
        () -> claim.processClaim(null));
}
```

```
@Test
@DisplayName("calculatePayout trả về 85% khi Approved")
void calculatePayout_StatusApproved_Returns85Percent() {
    // Arrange
    claim.processClaim("Approved");

    // Act
    double payout = claim.calculatePayout();

    // Assert
    assertEquals(850.0, payout, 0.001);
}

@Test
@DisplayName("calculatePayout trả về 0 khi không phải Approved")
void calculatePayout_StatusNotApproved_ReturnsZero() {
    assertEquals(0, claim.calculatePayout());
}

@Test
@DisplayName("updateClaimAmount cập nhật thành công")
void updateClaimAmount_ValidValue_Updates() {
    // Act
    claim.updateClaimAmount(2000.0);

    // Assert
    assertEquals(2000.0, claim.getAmount());
}

@Test
@DisplayName("updateClaimAmount ném exception khi <= 0")
void updateClaimAmount_Invalid_Throws() {
    assertThrows(IllegalArgumentException.class,
        () -> claim.updateClaimAmount(0));
}

@ParameterizedTest(name = "status={0} → payout={1}")
@CsvSource({
    "Approved, 850.0",
    "Rejected, 0",
    "Pending, 0"
})
@DisplayName("Parameterized: calculatePayout cho các trạng thái khác nhau")
void calculatePayout_VariousStatuses(String status, double expectedPayout) {
    // Arrange
    if (!"Pending".equals(status)) {
        claim.processClaim(status);
    }

    // Act
    double payout = claim.calculatePayout();

    // Assert
    assertEquals(expectedPayout, payout, 0.001);
}
```

```

    }

    @Test
    @DisplayName("toString trả về đúng định dạng")
    void toString_ReturnsExpectedFormat() {
        // Act
        String output = claim.toString();

        // Assert
        assertEquals(
            () -> assertTrue(output.contains("InsuranceClaim")),
            () -> assertTrue(output.contains("claimId='C001'")),
            () -> assertTrue(output.contains("amount=1000.0")),
            () -> assertTrue(output.contains("claimStatus='Pending'"))
        );
    }
}

```

Bước 3: Chạy mvn test hoặc Run trong IntelliJ → tất cả test phải Pass 

Phần VII – Bài tập nộp: AccountService

7.1. Yêu cầu Lab 2: Viết unit test cho chức năng Đăng ký tài khoản (Sẽ nộp bài trên Github classroom, chấm điểm tự động) Tham khảo trước nội dung:

Class AccountService có ít nhất 2 phương thức:

- boolean registerAccount(String username, String password, String email)
- boolean isValidEmail(String email)

Quy tắc nghiệp vụ (Business Rules):

Trường	Quy tắc
username	Không null, không rỗng
password	Độ dài lớn hơn 6 ký tự
email	Đúng định dạng RFC (có @ và domain)

7.2. Class AccountService

```

package tralTB.example;

import java.util.regex.Pattern;

public class AccountService {

    private static final Pattern EMAIL_PATTERN =
        Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$");
}

```

```
public boolean isValidEmail(String email) {
    if (email == null || email.isBlank()) {
        return false;
    }
    return EMAIL_PATTERN.matcher(email).matches();
}

public boolean registerAccount(String username, String password, String
email) {
    if (username == null || username.isBlank()) return false;
    if (password == null || password.length() <= 6) return false;
    if (!isValidEmail(email)) return false;
    // (Demo) – bỏ qua phần lưu DB
    return true;
}
}
```

7.3. File dữ liệu test

```
username,password,email,expected
john123,pass123,john@example.com,true
,pass123,john@example.com,false
alice,short,alice@mail.com,false
bob123,password,bobmail.com,false
carol,password,carol@domain.com,true
```

Giải thích các bộ test:

Dòng	Lý do
1	Hợp lệ → true
2	username rỗng → false
3	password chỉ 5 ký tự → false
4	email không có @ → false
5	Hợp lệ → true

7.4. Class test AccountServiceTest

```
package tralTB.example;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvFileSource;
import org.junit.jupiter.params.provider.CsvSource;
import org.junit.jupiter.params.provider.ValueSource;

import static org.junit.jupiter.api.Assertions.*;
```

```
class AccountServiceTest {

    private AccountService service;

    @BeforeEach
    void setUp() {
        service = new AccountService();
    }

    // ----- isValidEmail -----

    @ParameterizedTest(name = "Email hợp lệ: {0}")
    @ValueSource(strings = {
        "john@example.com",
        "alice.b@mail.co.uk",
        "carol_99@domain.io"
    })
    @DisplayName("isValidEmail trả về true với email đúng định dạng")
    void isValidEmail_ValidEmails_ReturnsTrue(String email) {
        // Act
        boolean result = service.isValidEmail(email);

        // Assert
        assertTrue(result);
    }

    @ParameterizedTest(name = "Email không hợp lệ: \"{0}\"")
    @CsvSource(value = {
        "bobmail.com",           // thiếu @
        "missing@dot",           // thiếu .domain
        "'@nodomain.com'",       // thiếu local part
        "' '",                   // chỉ khoảng trắng
        "NULL"                   // sẽ map về null
    }, nullValues = "NULL")
    @DisplayName("isValidEmail trả về false với email sai định dạng / null")
    void isValidEmail_InvalidEmails_ReturnsFalse(String email) {
        assertFalse(service.isValidEmail(email));
    }

    // ----- registerAccount (CSV File Source) -----

    @ParameterizedTest(name = "Row {index}: ({0},{1},{2}) → {3}")
    @CsvFileSource(resources = "/test-data.csv", numLinesToSkip = 1)
    @DisplayName("registerAccount với dữ liệu từ test-data.csv")
    void registerAccount_FromCsv(String username, String password,
        String email, boolean expected) {
        // Act
        boolean actual = service.registerAccount(username, password, email);

        // Assert
        assertEquals(expected, actual,
            () -> String.format("(%s,%s,%s) phải trả về %s",
                username, password, email, expected));
    }

    // ----- Các edge case bổ sung -----
}
```

```

@Test
@DisplayName("registerAccount: password = 6 ký tự (biên dưới) → false")
void registerAccount_PasswordExactly6_ReturnsFalse() {
    // Arrange
    String username = "bob";
    String password = "abcdef";           // đúng 6 ký tự
    String email = "bob@mail.com";

    // Act
    boolean actual = service.registerAccount(username, password, email);

    // Assert
    assertFalse(actual, "password phải > 6 ký tự");
}

@Test
@DisplayName("registerAccount: password = 7 ký tự (biên trên) → true")
void registerAccount_PasswordExactly7_ReturnsTrue() {
    assertTrue(service.registerAccount("bob", "abcdefg",
"bob@mail.com"));
}

@Test
@DisplayName("registerAccount: tất cả tham số null → false")
void registerAccount_AllNull_ReturnsFalse() {
    assertFalse(service.registerAccount(null, null, null));
}
}

```

7.5. Tiêu chí chấm điểm gợi ý

Tiêu chí	Trọng số
Cấu trúc dự án Maven đúng chuẩn (POM, thư mục)	10%
Class AccountService hiện thực đúng nghiệp vụ	20%
Test coverage $\geq 80\%$ (đo bằng JaCoCo)	20%
Áp dụng đầy đủ AAA Pattern (Arrange/Act/Assert rõ ràng)	20%
Có Parameterized Test đọc từ CSV	15%
Có test cho edge case & null	15%

7.6. (Khuyến nghị) Thêm JaCoCo để đo coverage

```

<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.11</version>
  <executions>
    <execution>

```

```

        <goals><goal>prepare-agent</goal></goals>
    </execution>
    <execution>
        <id>report</id>
        <phase>test</phase>
        <goals><goal>report</goal></goals>
    </execution>
</executions>
</plugin>

```

Chạy lệnh sau để sinh báo cáo độ phủ:

```
mvn test
```

Mở <target/site/jacoco/index.html> để xem báo cáo độ phủ.

Phần VIII – Best Practices & Checklist

8.1. Bảng tổng kết lưu ý khi lập trình JUnit

Chủ đề	Lưu ý
@Test	Đánh dấu phương thức kiểm thử
assertEquals, assertThrows, ...	So sánh kết quả mong đợi và thực tế
Isolation (cô lập)	Mỗi phương thức test chỉ kiểm thử một logic duy nhất
Test Naming	Tên hàm phải mô tả rõ mục đích – mẫu methodName_state_expected
Setup chung	Dùng @BeforeEach khi nhiều test dùng chung khởi tạo
Test Coverage	Bao phủ đủ các case: normal, edge, error
Không test logic trong test	Không lặp lại logic cần kiểm thử trong chính test (tránh "test sai chứng minh code sai")

8.2. Checklist trước khi nộp Lab 2

- Project chạy được với JDK 17 + Maven.
- pom.xml có đủ junit-jupiter-api, junit-jupiter-engine, junit-jupiter-params.
- Có class AccountService hiện thực registerAccount và isValidEmail.

- Có file src/test/resources/test-data.csv với ít nhất 5 dòng dữ liệu.
- Tất cả test class đều có @DisplayName mô tả rõ ràng.
- Mỗi test rõ 3 khối **Arrange / Act / Assert**.
- Có ít nhất 1 test cho normal case, 1 cho edge case, 1 cho error case.
- mvn test → BUILD SUCCESS, **không có test nào fail**.
- (Optional) JaCoCo coverage $\geq 80\%$.